CS230

Predictive Modeling of NFL Player Fantasy Points Using Deep Learning and Traditional ML

Evan Ellington
Department of Computer Science
Stanford University
eellingt@stanford.edu

Abstract

Fantasy Football involves drafting a slew of players in varying positions to comprise your team for that year's season. The way in which one drafts their team is a crucial part of the overall strategy of fantasy football. Week to week during the regular season, your team is matched up against others in your league and you compete to see whose team gets the most fantasy points. The points come from individual players' performances and is therefore critically impacted by one's strategy during the draft. In this paper, I will explain six different models I implemented with the goal of predicting the amount of fantasy points each player will get in a season.
Link to code: https://github.com/eellington3/CS230-Deep-Learning

1 Introduction

Millions of sports fans create fantasy football teams every year; many create multiple teams across different leagues and a lot of prizes and punishments are wagered for each team that is created. The goal of my research was to model draft strategies that maximize the projected points fantasy football teams can achieve. The input is a set of data frames, one for each position on a fantasy football team - quarterback (QB), running back (RB), wide receiver (WR), tight end (TE), kicker (K), and defense (DST). The output of each model predicts the fantasy points in the upcoming season for each player. I compared the resulting output between an Individual Positional NN, a Multi-Positional NN, a Support Vector Regression Model (SVR), a Random Forest Regression Model (RFR), a Gradient Boosting Regression Model (GBR), and an XGBoost Model (XGB).

2 Related work

Most fantasy football draft strategies take into account several different stats, opinions or news from various sources, individual preference, etc. By the time all teams have been drafted, every player has a new stat called ADP. ADP is the average draft position of each individual player. It represents the order each player typically was selected that year. The closer a player's ADP is to 1, means that overall, across all fantasy team drafts, teams who had the first pick of the draft most often took that player. APD of previous years and predicted ADP is a statistic that heavily drives most real-world draft strategies [1]. For simulating a real-world draft where each model acted as one team in the league, I assigned the opponents of each model to draft based on ADP. This caused the simulated opponents to draft players that on average, were the most common choice in the real world at that pick in the draft.

There is no single widely accepted model for predicting fantasy football points, but several approaches are commonly used. Support Vector Regression (SVR) effectively models nonlinear relationships

CS230: Deep Learning, Winter 2018, Stanford University, CA. (LateX template borrowed from NIPS 2017.)

between player statistics and fantasy points, even with limited data, and has demonstrated reasonable accuracy across multiple positions [2]. Random Forest Regression (RFR) handles high-dimensional data, captures complex interactions, and reduces overfitting through averaging, making it practical for player performance prediction [3]. Gradient Boosting Regression (GBR) incrementally corrects errors and captures subtle patterns in noisy NFL data, showing strong predictive performance for player-level projections [4] [5]. XGBoost, in particular, has been applied to generate weekly fantasy point forecasts, highlighting its effectiveness in real-world applications [6] [7].

3 Dataset and Features

My data consisted of positional data from the 2023, 2024, and some of ongoing 2025 NFL season. I compiled the data from different online sources [9] [10]. For each of the NFL seasons in my data, I had a data frame for each position each containing position-specific statistics on 32-100 players in that position. I used a train:test split of 80%:20% or 85%:15% depending on the model and used feature scaling for each model since across positional datasets, the raw values widely vary in range. See Tables 4- 9 in Appendix for an example top couple rows of each position data set.

4 Methods

I built 2 separate implementations of neural networks for predicting 2025 fantasy football points. One compiled the predicted points for each player based on the results from 6 different neural networks, one for each position (QB, RB, WR, TE, K ,DST), this is the Individual Positional NN. The other implementation predicted points for each player using one neural network that looked at all data across all positions, the Multi-Positional NN. Additionally, I implemented an SVR Model, an RFR Model, a GBR Model, and an XGB Model as a result of my reasearch.

All six models were trained and tested with the same logic besides from some slight variation in the train:test split. The models were trained by looking at 2023 season data and using what it could learn from there to predict a player's fantasy points in 2024. I then tuned any hyperparameters based on the performance of the predictions. I then used the trained model to evaluate the 2024 season and predict 2025 season player stats which is what was used in the draft simulation logic.

To simulate how these models would perform in a real fantasy football league, I designed a snake draft (draft order goes $1 \rightarrow 12$ then $12 \rightarrow 1$) for a typical league of 12 teams. In each draft, the associated model was assigned a random draft pick, and the rest of the picks, the opponents, were set to use the draft strategy to draft on real-world ADP.

5 Results

5.1 Individual Positional Neural Network

To build the individual position neural networks, I found that the positional data sets had two general structures. The QB, RB, WR, and TE data sets were relatively similar, meaning that they had a comparable number of total players and a roughly similar number of features. The K and DST data sets were also similar to each other but far smaller than those of QB, RB, WR, or TE. Due to this distinguishable difference, I implemented separate neural network architectures for K and DST positions versus QB, RB, WR, and TE.

The network used for QB, RB, WR, and TE was a fully connected neural network with three hidden layers containing 16, 8, 4 nodes and a single output layer for predicting fantasy points. I chose a train:test split of 85%:15%, as using less than 15% for testing would have left too little data to sufficiently evaluate the model on unseen players. The Adam optimizer was selected for its effectiveness on noisy data, which was appropriate given the high variability in player performance both across players and across seasons. A dropout rate of 0.3 was used to improve the model's ability to generalize and predict future-year performance. Since these positional data sets were relatively small, early stopping was implemented to mitigate overfitting.

For the K and DST positions, the smaller size of the data sets motivated a simpler network architecture. The network consisted of two fully connected hidden layers with 16 and 8 neurons respectively,

followed by a single output layer. The same 85%:15% train:test split was used, and early stopping was applied. The model was compiled with the Adam optimizer, consistent with the other positional networks.

5.2 Multi-Positional Neural Network

For the multi-positional model, all player positions were combined into a single dataset, allowing the neural network to learn patterns both within and across positions. The network consisted of two fully connected hidden layers with 32 and 16 neurons, respectively, using ReLU activation, followed by a dropout layer with a rate of 0.4 and a single linear output layer for predicting fantasy points. L2 regularization was applied to the weights of both hidden layers to penalize excessively large weights, helping to prevent overfitting by encouraging the network to learn simpler, more generalizable patterns across positions. A train:test split of 85%:15% was used, consistent with the individual positional models, to ensure sufficient data for both training and evaluation. The Adam optimizer was chosen with a learning rate of 0.0005 to handle the noisy, high-variance nature of the data. Early stopping with a patience of 50 epochs was implemented to monitor validation loss and restore the best weights, further reducing the risk of overfitting.

5.3 Support Vector Regression Model

I implemented the SVR model with an RBF (Radial Basis Function) kernel, a regularization parameter C=10, and an epsilon of 0.5. The RBF kernel allows the model to capture the nonlinear relationships between player statistics and fantasy points, such as interactions between yards, touchdowns, fumbles, etc. The regularization parameter C balances fitting the training data closely with maintaining model smoothness, after some trial and error, I found a value of 10 allows the model to capture some patterns without excessive overfitting. The epsilon parameter defines a tolerance margin around the predicted values, so deviations smaller than 0.5 points are ignored, reducing sensitivity to minor variability in player performance. These choices together allowed the SVR model to best capture trends in the fantasy points while attempting to ignore noise in the data.

5.4 Random Forest Regression Model

I then implemented the RFR model with 200 decision trees. The number of trees determines how many individual decision trees are trained and combined, 200 trees seemed to best provide a balance between stable, accurate predictions and computational efficiency. Each tree can capture different interactions among player statistics, such as yards, touchdowns, and ADP, and averaging across all trees reduces sensitivity to any single noisy observation.

5.5 Gradient Boosting Regression Model

I used a GBR Model with 500 boosting stages, a learning rate of 0.05, and a maximum tree depth of 3. The number of boosting stages determines how many sequential trees are trained, with each tree correcting the errors of the previous one. 500 trees allowed the model to gradually improve predictions while balancing bias and variance. The learning rate controls the contribution of each tree, and a value of 0.05 helped prevent overfitting to noisy aspects in the data. The maximum depth of 3 ensures that each tree remains shallow, capturing simple interactions such as yards and receptions without overfitting to rare events, like a hail mary pass for a touchdown. These choices enable the GBR model to capture meaningful patterns in player performance while remaining resistant to variability in fantasy scoring.

5.6 XGBoost Regression Model

Finally, I implemented an XGB Regressor Model with 600 boosting rounds, a learning rate of 0.035, maximum tree depth of 5, a subsample of 0.8, column subsampling of 0.8, L2 regularization with $\lambda = 1.2$, and a squared error loss function. The high number of boosting rounds allows the model to gradually improve predictions by sequentially correcting errors, which is important given the noisy positional data. The low learning rate ensures that each tree makes small incremental adjustments After some trial and error, the learning rate of 0.035 seemed to best reduce overfitting to small fluctuations in points. Limiting the tree depth to 5 captures interactions such as yards -

touchdowns - receptions while avoiding overfitting to rare cases, like a hail mary touchdown pass for 70 yards. Subsampling 80% of both rows and features for each tree adds regularization, encouraging generalization and preventing reliance on any single statistic. I found L2 regularization with $\lambda = 1.2$ to better prevent the model from overfitting compared to $\lambda = 1$. All together, these choices allow the XGBoost model to accurately capture meaningful patterns in player performance while remaining stable against variability in the data.

All of these models were validated using MSE as the loss function, the resulting RMSE values can be seen below in Table 1. MSE was used as the loss function to more heavily penalize larger deviations, which is important because high-performing players have the greatest impact on total fantasy points.

Table 1: Model RMSE Value for Each Position and Average/Overall

	QB	RB	WR	TE	K	DST	Avg RMSE
Ind. Pos NN	121.143	73.907	42.571	32.498	17.099	35.039	53.709
Multi-Pos. NN	-	-	-	-	-	-	65.162
SVR Model	162.143	79.782	63.900	44.444	12.194	21.525	63.998
RFR Model	95.317	30.523	30.567	14.603	7.927	12.332	31.878
GBR Model	104.989	28.079	29.334	11.833	10.741	4.744	31.620
XGB Model	102.384	27.104	29.712	15.589	12.409	8.920	32.686

5.7 Performance of Models in Draft

To put these various model predictions to the test, for each model I simulated 100 snake drafts (the typical drafting configuration for fantasy football leagues) and had the model act as one randomly selected draft pick and the other 11 picks in the draft, the opponents, were drafting based on ADP. In the draft logic, I ensured that each team drafts a full roster, meaning that they have at least one pick for each position and added an upper bound to positions to enforce real-world strategy, like not having 5 quarterbacks and one wide receiver. To compare performance, I computed the total amount of actual fantasy points for each team that was generated. The average scores from these simulations can be seen in Table 3 along with average win margin, and the gap between the model and oppenent score distributions (a visualization of the distributions and overlap of scores for each model can be seen in Figure 2 in the Appendix). On average, every model's average total score beat out that of the opponent. However, we can clearly see that some models did far better than others when we look at the margin at which they won by and by the distance, or rather lack of distance between the highest scoring opponent and the lowest scoring model.

To closely examine the differences between the models, I computed the R^2 value for each position in every model, except for the Multi-Positional NN, which just has one overall R^2 value since that was the one model that was not separated by position. The closer the R^2 value is to 1, the more accurate the model, where as the closer the R^2 value is to 0, the model's prediction is no better than taking the overall average as the prediction, and if it's less than 0, then the model's predictions are worse than taking the average as the prediction. Table 2 displays of all the R^2 values for each position across all different models as well as their averages (a visualization of the R^2 values as a heat map is shown in Figure 3 in the Appendix).

We can see from the plot of average R^2 values vs the score gap (the last row in Table 3) between each of the models with its respective opponents in Figure 1. This shows a positive linear relationship between distribution score gap and average R^2 value for the model. There is one outlier, which is the Individual Positional NN. The Individual Positional NN Model has an abnormally low R^2 value for the amount of overlap between the model's score distribution and the opponent's score distribution. This is likely because the model is predicting very poorly on kickers and defense positions which is bringing its average R^2 down. While it is predicting much better on the more impactful positions, QB, RB, WR, and TE. These positions are more impactful because on average they score far more points than K and DST positions. So accurately or inaccurately predicting the points for kickers and defense has far less of an impact on the model's score distribution compared to predicting points for quarterbacks, running backs, wide receivers, and tight ends. Focusing on the most impactful positions in the top performing models, those being RFR, XGB, and GBR, we can see that these models are accurately predicting points for the RB, WR, and TE positions, and less so for the QB positions.

This is likely due to the inherent values in the data. RB, WR, and TE positions have less variance in their respective data sets compared to the QB dataset, so it makes sense that these models are more accurately predicting fantasy points for those positions compared to the QB position.

Table 2: Model R^2 value Across Positions and Average/Overall R^2 Value

	QB	RB	WR	TE	K	DST	Avg Model R^2 Value
Ind. Pos NN	0.515	0.412	0.513	0.763	-0.181	-0.029	0.332
Multi-Pos. NN	-	-	-	-	-	-	0.619
SVR Model	0.058	0.426	0.324	0.514	0.794	0.275	0.399
RFR Model	0.674	0.916	0.845	0.948	0.913	0.762	0.801
GBR Model	0.605	0.929	0.857	0.966	0.840	0.965	0.860
XGB Model	0.624	0.934	0.854	0.940	0.787	0.875	0.836

Table 3: Average Score Over 100 iterations for Each Model and its Respective Opponents

	Ind. Pos NN	Multi-Pos. NN	SVR	RFR	GBR	XGB
Model	3645.29	3587.41	3104.42	3673.49	3683.82	3682.05
Opponent	2715.67	2722.50	2687.39	2650.15	2648.47	2644.80
Average Win Margin ($ModelAVG - OppAVG$)	+929.62	+864.91	+417.03	+1023.34	+1035.35	+1037.24
Gap b/w Model and Opp ($lowestModel - highestOpp$)	-27.2	-183.7	-494.2	+259.2	+272.2	+192.2

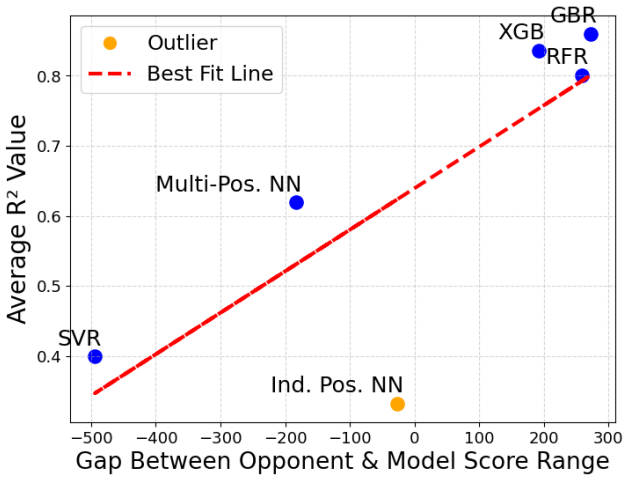


Figure 1: Plot of model performance against opponents vs the average/overall R^2 value of the model.

6 Conclusion

The highest performing model was the Gradient Boosting Regression Model. Although both of the neural network models outperformed the SVR model, which is a commonly used model for predicting fantasy points, it is likely that the amount of data that was available to me for this project was insufficient to fully leverage the potential power of the neural networks. I hope to continue to maintain the datasets that I compiled for this project and retrain my neural network models with many more seasons worth of data in the future to be able to better learn trends and achieve performance equal to or greater than that of the Gradient Boosting Regression Model.

References

[1] Murfet, A. (2023). Fantasy Football Draft Strategy Advice: Average Draft Position (ADP) Guide. FantasyPros. Available at: https://www.fantasypros.com/2023/08/fantasy-football-draft-strategy-advice-average-draft-position-adp-guide/

[2] Smola, A. Schölkopf, B. (2015). A Machine Learning Approach to Fantasy Football Score Prediction Using Support Vector Regression. arXiv preprint arXiv:1505.06918. Available at: https://arxiv.org/abs/1505.06918

[3] ffverse (ffopportunity) (2023–2025). ffopportunity: Models for Fantasy Football Expected Points (XGBoost models and utilities). ffverse GitHub repository. Available at: https://github.com/ffverse/ffopportunity

[4] Bratkovics, C. (2023). fantasy-football-ai: Production ML system for fantasy forecasting (ensemble includes XGBoost). GitHub repository. Available at: https://github.com/cbratkovics/fantasy-football-ai

[5] Wu, D. (2025). Predictive Analytics for NFL Pick'em Contests: A Comparative Study of Gradient Boosting Models. UCLA Electronic Theses and Dissertations. Available at: https://escholarship.org/uc/item/9x33j0fz

[6] Nathan Knows Blog (2024–2025). The Predictive Playbook — Weekly QB Fantasy Point Projections (XGBoost used for QB projections). Nathan Knows Blog. Available at: https://nathanknowsblog.com/tag/nfl/

[7] King, J. (2020). Using Machine Learning to Predict Fantasy Football Points: A Fantasy Football Trade Analyzer Using RNN-LSTM, ARIMA, XGBoost and Dash. Medium. Available at: https://medium.com/data-science/using-machine-learning-to-predict-fantasy-football-points-72f77cb0678a

[8] TensorFlow Developers. (2023). TensorFlow: An end-to-end open source machine learning platform. Available at: https://www.tensorflow.org/.

[9] FantasyPros. (2025). NFL Average Draft Position (ADP) — Overall. FantasyPros. Available at: https://www.fantasypros.com/nfl/adp/overall.php (accessed Dec 5, 2025).

[10] CBS Sports. (2025). 2024 NFL Fantasy Football QB PPR Stats. CBS Sports. Available at: https://www.cbssports.com/fantasy/football/stats/QB/2024/season/stats/ppr/ (accessed Dec 5, 2025).

7 Appendix

Table 4: Example QB Data from 2023 Season

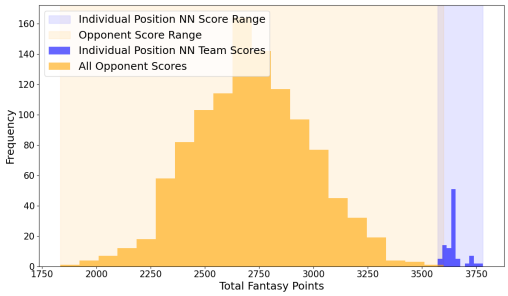
Name_pos_team	Team	Name	Pos	Age	GP	Att	Comp	PYds	Y/G	PTD	INT	Rate	RAtt	RYds	Y/A	RTD	FL	FPts	FP/G	ADP
Josh Allen QB BUF	BUF	Josh Allen	QB	27	17	579	385	4306	253.3	29	18	92.2	111	524	4.7	15	4	434	25.5	18.5
Dak Prescott QB DAL	DAL	Dak Prescott	QB	30	17	590	410	4516	265.6	36	9	105.9	55	242	4.4	2	2	400	23.5	88
Jalen Hurts QB PHI	PHI	Jalen Hurts	QB	25	17	538	352	3858	226.9	23	15	89.1	157	605	3.9	15	5	388	22.8	21.5
Jordan Love QB GB	GB	Jordan Love	QB	24	17	579	372	4159	244.6	32	11	96.1	50	247	4.9	4	3	370	21.8	163
Lamar Jackson QB BAL	BAL	Lamar Jackson	QB	26	16	457	307	3678	229.9	24	7	102.7	148	821	5.6	5	6	364	22.8	32.5

Table 5: Example RB Data from 2023 Season

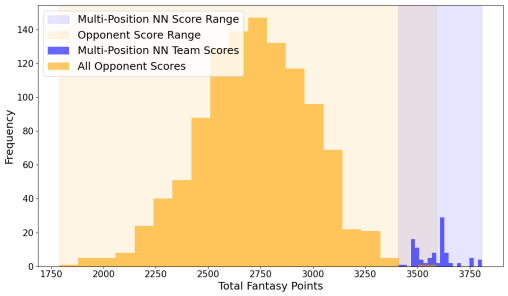
Name_pos_team	Name	Pos	GP	RushAtt	RushYds	Y/A	RushTD	Tgt	Rec	RecYds	Y/G	Y/Rec	RecTD	FumLost	FPts	FPts/G	ADP
Christian McCaffrey RB SF	Christian McCaffrey	RB	16	272	1459	5.4	14	83	67	564	35.3	8.4	7	2	391.3	24.5	1.5
Breece Hall RB NYJ	Breece Hall	RB	17	223	994	4.5	5	95	76	591	34.8	7.8	4	—	290.5	17.1	46.5
Travis Etienne RB JAC	Travis Etienne	RB	17	267	1008	3.8	11	73	58	476	28	8.2	1	—	282.4	16.6	26
Rachaad White RB TB	Rachaad White	RB	17	272	990	3.6	6	70	64	549	32.3	8.6	3	2	267.9	15.8	62.5
Raheem Mostert RB LV	Raheem Mostert	RB	15	209	1012	4.8	18	32	25	175	11.7	7	3	1	267.7	17.8	115

Table 6: Example WR Data from 2023 Season

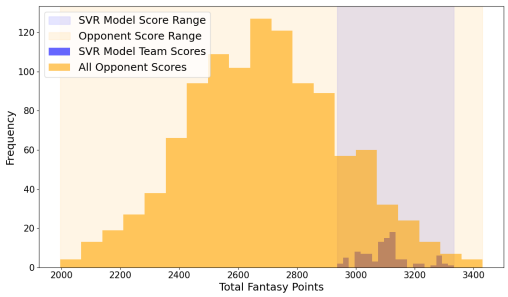
Name_pos_team	Name	Pos	GP	Tgt	Rec	RecYds	Y/G	Y/Rec	RecTD	RushAtt	RushYds	Y/A	RushTD	FumLost	FPts	FPts/G	ADP
CeeDee Lamb WR DAL	CeeDee Lamb	WR	17	181	135	1749	102.9	13	12	14	113	8.1	2	2	255	15	12
Tyreek Hill WR MIA	Tyreek Hill	WR	16	171	119	1799	112.4	15.1	13	6	15	2.5	—	1	250	15.6	6.5
Amon-Ra St. Brown WR DET	Amon-Ra St.	WR	16	164	119	1515	94.7	12.7	10	4	24	6	—	1	204	12.8	18.5
Mike Evans WR TB	Mike Evans	WR	17	136	79	1255	73.8	15.9	13	—	—	0	—	—	197	11.6	77.5
Puka Nacua WR LAR	Puka Nacua	WR	17	160	105	1486	87.4	14.2	6	12	89	7.4	—	—	183	10.8	275.5



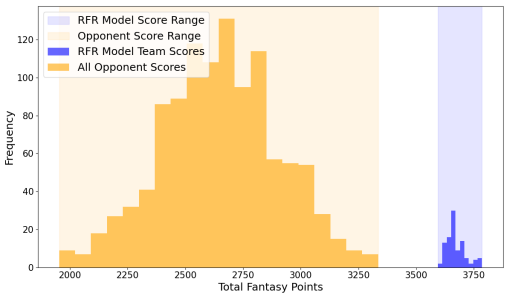
(a) Individual Positional NN Team



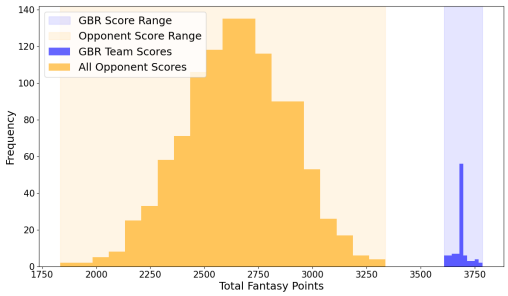
(b) Multi-Positional NN Team



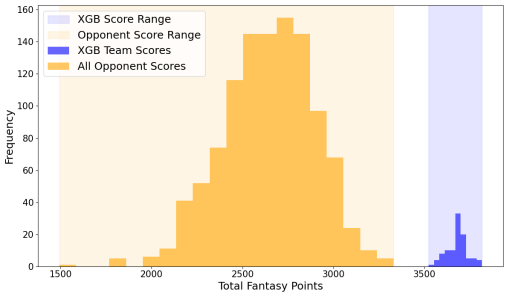
(c) SVR Model Team



(d) RFR Model Team



(e) GBR Model Team



(f) XGB Model Team

Figure 2: Comparison of 6 different model score distributions against league opponent teams across 100 drafts.

Table 7: Example TE Data from 2023 Season

Name_pos_team	Name	Pos	GP	Tgt	Rec	RecYds	Y/G	Y/Rec	RecTD	FumLost	FPts	FPts/G	ADP
Sam LaPorta TE DET	Sam LaPorta	TE	17	120	86	889	52.3	10.3	10	—	239.3	14.1	152.5
Evan Engram TE DEN	Evan Engram	TE	17	143	114	963	56.6	8.5	4	2	230.3	13.5	85.5
Travis Kelce TE KC	Travis Kelce	TE	15	120	93	984	65.6	10.6	5	1	219.4	14.6	5
T.J. Hockenson TE MIN	T.J. Hockenson	TE	15	127	95	960	64	10.1	5	1	219	14.6	46.5
George Kittle TE SF	George Kittle	TE	16	90	65	1020	63.8	15.7	6	—	203.2	12.7	51.5

Table 8: Example K Data from 2023 Season

Name_pos_team	Name	Pos	GP	FGM	FGA	Long	FG20-29	FG20-29Att	FG30-39	FG30-39Att	FG40-49	FG40-49Att	FG50+	FG50+Att	XP	XPAtt	FPts	FPts/G	ADP
Brandon Aubrey K DAL	Brandon Aubrey	K	17	36	38	60	9	9	13	15	4	4	10	10	49	52	177	10.4	288
Matt Gay K WAS	Matt Gay	K	17	33	41	57	9	9	9	10	7	9	8	8	35	36	150	8.8	233.5
Cairo Santos K CHI	Cairo Santos	K	17	35	38	55	9	9	11	11	8	10	7	7	31	33	150	8.8	350
Justin Tucker K BAL	Justin Tucker	K	17	32	37	50	10	10	10	10	11	12	1	1	51	52	149	8.8	109.5
Jake Elliott K PHI	Jake Elliott	K	17	30	32	61	9	9	7	8	7	7	7	7	45	46	149	8.8	183.5

Table 9: Example DST Data from 2023 Season

Name	Pos	Int	Saf	Sacks	Tackles	DefFumRec	ForcedFum	DefTD	PtsAllowed	Pts/G	NetPassYdsAllowed	RushYdsAllowed	TotalYdsAllowed	Yds/G	FPts	FPts/G	ADP
Dallas Cowboys	DST	17	1	46	693	9	14	7	315	18.5	3481	1910	5095	299.7	242	14.2	127
Baltimore Ravens	DST	18	1	60	775	13	16	2	280	16.5	3717	1860	5123	301.4	236	13.9	160
NY Jets	DST	17	3	48	785	10	14	4	355	20.9	3182	2108	4969	292.3	224	13.2	154
Cleveland Browns	DST	18	0	49	753	10	14	3	362	21.3	3149	1793	4593	270.2	223	13.1	239.5
Buffalo Bills	DST	18	0	54	760	12	14	3	311	18.3	3676	1880	5222	307.2	222	13.1	139.5

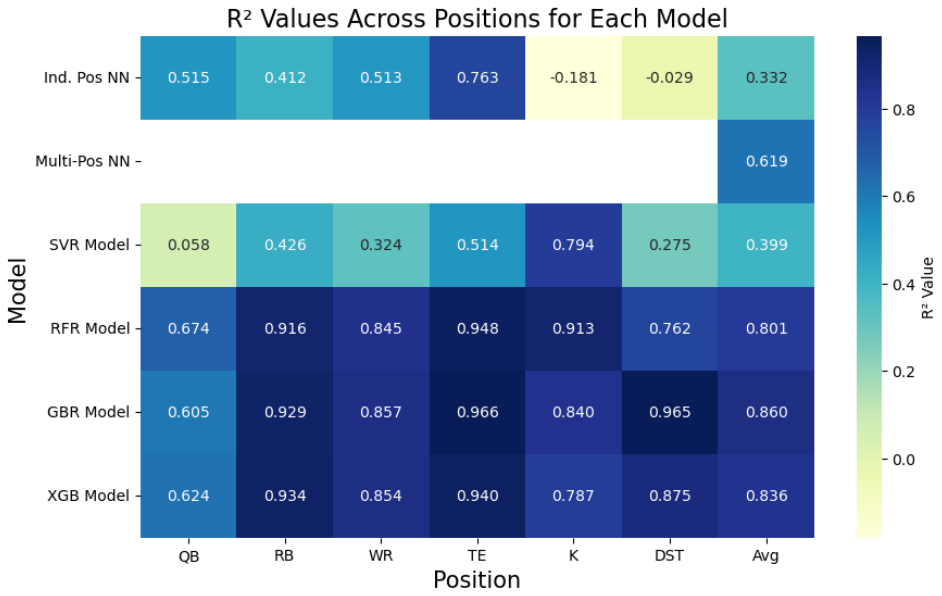


Figure 3: Heat Map of R^2 Values Across All Models per Position